Python Developer
Interview Preparation

Practical questions, model answers and pro notes - based on your resume

Build Your Skills | Crack Your Interview | Get Hired

What's Inside

- Python Fundamentals and OOP (page 2)
- Django, Flask and REST APIs (page 3)
- SQL and Databases (page 4)
- Your Resume Projects (page 5)
- Coding and HR Questions (page 6)
- Quick Revision Cheat Sheets (page 7)
- Project Architecture Diagrams (page 8)
- Final Tips and Checklists (page 9)

How to Use This Guide

1. Read each question and **answer aloud first**, then compare with the model answer.
2. Keep every answer to **30-60 seconds** and use simple words.
3. **Run every code sample** yourself - interviewers ask you to explain the output.
4. Connect each concept to **one of your own projects**.
5. Re-read the Quick Revision page (7) the night before.

7-Day Study Plan

- **Day 1** - Python basics, data structures, OOP, exceptions
- **Day 2** - Django: MVT, models, ORM, migrations, admin
- **Day 3** - DRF serializers, authentication, REST best practices, deployment on Render
- **Day 4** - SQL joins, indexing, transactions (ACID), query optimization
- **Day 5** - Walk through each resume project: flow, schema, challenges
- **Day 6** - Coding practice on HackerRank / LeetCode (easy + medium)
- **Day 7** - Mock interview, HR answers, revise the cheat sheets

Words to Remember

"The best way to predict your future is to create it."

- often attributed to Abraham Lincoln

Important Mindset Notes

- It is fine to say **"I'm not sure, but here is how I would find out."**
- Explain **why**, not just what - reasoning matters more than memorised lines.
- Never claim skills you cannot demonstrate in code or on a whiteboard.

1 Python Fundamentals & OOP

Basic concepts, data structures, functions, OOP and more.

Key Interview Questions + Model Answers

1. **What are the differences between lists and tuples?**
   Lists are mutable (can change), tuples are immutable. Tuples are slightly faster, and can be dictionary keys when their items are hashable.
2. **How do dictionaries work? Give an example.**
   A dict stores key-value pairs in a hash table, so lookup is O(1) on average. Example: user = {"name": "Asha"}; user["name"].
3. **What is the use of *args and **kwargs?**
   *args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dict.
4. **Explain OOP concepts: class, object, inheritance, polymorphism.**
   A class is a blueprint, an object is an instance. Inheritance reuses a parent class; polymorphism lets different classes respond to the same method name in their own way.
5. **What are generators? How are they different from lists?**
   Generators use yield and produce values lazily, one at a time, so they save memory. A list builds and stores every item up front.
6. **What is exception handling? Show a try-except example.**
   It handles runtime errors without crashing: try (risky code), except (handle), else (no error), finally (always runs).
7. **What is a virtual environment? Why is it used?**
   An isolated Python environment per project, so dependency versions never clash. Create with: python -m venv venv.

Sample Code

```
# List vs Tuple
lst = [1, 2, 3]
tup = (1, 2, 3)
lst.append(4)     # allowed
# tup.append(4)   # error: tuples are immutable

# try / except / finally
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Always runs")
```

Key Concepts

- Lists, tuples, sets, dicts
- Functions, lambda, scope
- Decorators and generators
- OOP: 4 pillars
- Exceptions and context managers
- Modules, packages, venv

Important Notes (Interview Favourites)

- **Mutable default argument trap:** def f(x, items=[]) shares one list across calls. Use items=None and create the list inside.
- **== vs is:** == compares values, "is" compares object identity (same object in memory).
- **GIL:** only one thread runs Python bytecode at a time. Use multiprocessing for CPU-heavy work; threads or async for I/O.
- **Shallow vs deep copy:** copy.copy copies the outer object only; copy.deepcopy copies nested objects too.
- **Know these:** @staticmethod, @classmethod, __init__, __str__, list comprehensions and the four pillars (encapsulation, abstraction, inheritance, polymorphism).
- **Tip:** link OOP to your project - every Django model inherits from models.Model.

2 Django, Flask & REST APIs

Web frameworks, MVT, serializers, APIs and security.

Interview Questions + Model Answers

1. **What is Django and what is MVT architecture?**
   Django is a batteries-included Python web framework. Model = data/DB, View = logic that handles requests, Template = HTML output. The URL router sends each request to a view.
2. **How does Django ORM work? Give an example query.**
   It maps Python classes to tables so you query without raw SQL, e.g. Product.objects.filter(price__lt=500).order_by("name"). Migrations keep the schema in sync.
3. **What is Django REST Framework? What is a serializer?**
   DRF is a toolkit for building APIs. A serializer converts model objects to JSON and back, and validates incoming data.
4. **Difference between Django and Flask.**
   Django is full-featured (ORM, admin, auth built in). Flask is a micro-framework: minimal core, you pick your own libraries.
5. **How do you secure a REST API?**
   Authentication (token/JWT/session), permission classes, HTTPS, input validation, throttling, correct CORS settings, no secrets in code.
6. **What is middleware in Django?**
   Hooks that process every request and response globally - security, sessions, authentication, CORS, logging.
7. **How do you deploy a Django/Flask app?**
   Set DEBUG=False and ALLOWED_HOSTS, use env variables, run with gunicorn, serve static files (WhiteNoise), connect PostgreSQL, then deploy on a host such as Render.

Example: View + Serializer

```
# views.py
from django.http import JsonResponse

def home(request):
    data = {"message": "Hello from Django"}
    return JsonResponse(data)

# serializers.py
from rest_framework import serializers

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price"]
```

Key Topics

- MVT architecture
- Serializers
- Authentication
- CRUD operations
- API security
- Deployment (Render)

Important Notes

- **Request flow:** URL -> middleware -> view -> ORM/DB -> serializer or template -> response.
- **N+1 queries:** use select_related (foreign keys) and prefetch_related (many-to-many / reverse) to cut database hits.
- **Status codes:** 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error.
- **Never** commit SECRET_KEY or passwords. Keep them in environment variables and set DEBUG=False in production.

3 SQL & Databases

Joins, indexing, transactions, optimization and more.

Interview Questions + Model Answers

1. **What are different types of joins? Explain with example.**
   INNER = only matching rows; LEFT = all left rows + matches; RIGHT = all right rows + matches; FULL = all rows from both sides.
2. **What is normalization? Why is it important?**
   Organising tables (1NF, 2NF, 3NF) to remove duplicate data and update anomalies, which keeps data consistent.
3. **How does indexing improve performance?**
   An index (usually a B-tree) lets the database find rows without scanning the whole table. Trade-off: slower writes and extra storage.
4. **What are transactions? Explain ACID properties.**
   A transaction groups statements into one unit. Atomicity (all or nothing), Consistency (rules stay valid), Isolation (parallel transactions do not interfere), Durability (committed data survives crashes).
5. **How to prevent SQL injection?**
   Use parameterized queries or the ORM and never build SQL by joining user input into strings. Validate input and use least-privilege DB users.
6. **What is a view? What is a materialized view?**
   A view is a saved query that acts like a virtual table. A materialized view stores the result physically and must be refreshed.
7. **Difference between WHERE and HAVING?**
   WHERE filters rows before grouping; HAVING filters groups after aggregation (COUNT, SUM, ...).

Example Queries

```sql
-- Students and their courses
SELECT s.name, c.course_name
FROM students s
JOIN courses c ON s.course_id = c.id
WHERE s.age > 18;

-- Courses with more than 5 students
SELECT c.course_name, COUNT(*) AS total
FROM students s
JOIN courses c ON s.course_id = c.id
GROUP BY c.course_name
HAVING COUNT(*) > 5
ORDER BY total DESC;

-- Add an index
CREATE INDEX idx_students_course
ON students (course_id);
```

Key Topics

- Joins (INNER, LEFT, RIGHT, FULL)
- Indexing and query optimization
- Transactions and ACID
- SQL injection prevention
- Views and materialized views
- Keys: primary, foreign, unique

Important Notes

- **Execution order:** FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT.
- **Slow query?** Run EXPLAIN ANALYZE (PostgreSQL), index columns used in WHERE / JOIN / ORDER BY, avoid SELECT *, paginate.
- **DELETE vs TRUNCATE vs DROP:** DELETE removes chosen rows (can roll back); TRUNCATE empties the table; DROP removes the table itself.
- **In Django:** wrap related writes in transaction.atomic(); the ORM parameterizes queries automatically.

4 Your Resume Projects

Project-wise questions with talking points to prepare.

Pics - E-Commerce (Django + DRF + PostgreSQL)

- **How does your product catalogue work?** Category and Product models (price, stock, image); list/detail API endpoints with filtering and pagination.
- **What is the role of DRF serializers?** Convert model objects to JSON, validate incoming data, control which fields are exposed.
- **How did you handle user authentication and cart?** Token/JWT login; Cart and CartItem linked to the user; permissions so users only see their own cart.
- **How did you deploy it on Render?** gunicorn start command, environment variables, PostgreSQL URL, static files via WhiteNoise, DEBUG=False.

LWT - Learning Platform (Django + DRF)

- **How did you manage video uploads and study material?** File fields with media storage, material linked to courses, file type and size validation.
- **Explain the career guidance module.** Be ready to explain what data it uses, how guidance is shown, and why you designed it that way.
- **How did you implement user roles (student/teacher)?** Role field or groups plus custom permission classes; only teachers can upload.

VBI - Virtual Bank of India (Django + PostgreSQL)

- **How does the transaction system work?** A transfer runs inside transaction.atomic(): debit, credit and the transaction record all succeed or all roll back.
- **How did you maintain balance accuracy?** select_for_update() row locking, DecimalField (never float), checks for negative or insufficient funds.
- **What measures did you take for security?** Authentication, hashed passwords, permission checks, HTTPS, input validation, an audit log of transactions.

Important Notes: How to Present Your Projects

- **Use this 5-step answer:** Problem -> Tech stack -> Your role -> One real challenge and how you solved it -> Result.
- **Prepare a 60-second pitch** for each project and be able to draw its database tables and relationships on paper.
- **Be honest:** the hints above are suggested talking points - adjust them to what you actually built, and explain only what you can defend.

5 Coding & Practical Questions

Common coding problems and real-time scenarios.

Python Coding Questions

```python
# 1. Sum of even numbers up to n
n = int(input())
print(sum(i for i in range(1, n + 1) if i % 2 == 0))

# 2. Fibonacci series (first n terms)
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b

# 3. Factorial
def factorial(n):
    return 1 if n <= 1 else n * factorial(n - 1)

# 4. Palindrome check
def is_palindrome(s):
    s = s.lower()
    return s == s[::-1]

# 5. Reverse a string
print("python"[::-1])   # nohtyp

# 6. File handling
with open("data.txt") as f:
    for line in f:
        print(line.strip())
```

Debugging Scenarios

- **Fix the error:** read the traceback from the bottom up, reproduce, fix, re-run.
- **Slow SQL query:** EXPLAIN ANALYZE, add an index, avoid SELECT *, use select_related.
- **API exceptions (Django/Flask):** use try/except, return proper status codes and log the error.
- **Memory leaks:** look for ever-growing globals/caches and unclosed files; use generators; profile with tracemalloc.

HR / Behavioural Questions: How to Answer

- **Tell me about yourself:** background -> skills and projects -> why this role (about 60 seconds).
- **Why do you want to work here?** Research the company and connect its work to your skills.
- **Strengths and weaknesses:** a real strength with an example; a real weakness plus what you do to improve it.
- **Where do you see yourself in 5 years?** Growth in skills and responsibility - realistic, not vague.

Important Notes for Live Coding

- **Think aloud.** Clarify inputs and edge cases first (empty input, negatives, duplicates).
- **Start simple, then optimize.** Give the brute-force idea, then improve it.
- **Mention complexity** (time and space) and test with a small sample before saying you are done.
- **Also practise:** remove duplicates from a list, count word frequency, find the second largest number, two-sum.

6 Quick Revision - Key Topics

One glance, all important concepts.

Python

- Data structures
- OOP
- Exception handling
- Generators
- Decorators
- List comprehensions
- venv and pip

Django

- MVT
- ORM and migrations
- Serializers (DRF)
- Authentication
- Middleware
- Admin panel
- Settings and env vars

Flask

- Routing
- Templates (Jinja2)
- REST APIs
- Blueprints
- Deployment
- Request / response

SQL

- Joins
- Indexes
- Transactions (ACID)
- Optimization
- Normalization
- GROUP BY / HAVING

Git & GitHub

- Version control
- Branches
- Pull requests
- Collaboration
- Merge vs rebase
- .gitignore

Tools & Others

- Postman
- VS Code
- Render
- Pandas / NumPy
- Docker (nice to know)
- Logging and debugging

Django Command Cheat Sheet

```
django-admin startproject mysite
python manage.py startapp shop
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
python manage.py collectstatic
```

Git Cheat Sheet

```
git init
git clone <url>
git status
git add .
git commit -m "message"
git checkout -b feature-x
git merge feature-x
git push origin main
git pull
```

Important Notes: HTTP Methods

- **GET** read data
- **POST** create data
- **PUT** replace a record
- **PATCH** partial update
- **DELETE** remove a record

Important Notes: Night-Before Checklist

- Re-read all 7 model answers on pages 2-4
- Retype 3 coding solutions from memory
- Redraw your project architectures (page 8)
- Prepare your 60-second intro
- Sleep well - clarity beats cramming

7 Project Architecture Diagrams

High-level architecture of your main projects.

Pics - E-Commerce

1 **User** Browser → 2 **Frontend** HTML/CSS/JS → 3 **Django + DRF** API layer → 4 **PostgreSQL** Database → 5 **Render** Deploy

The frontend calls the DRF API; serializers validate data; the ORM reads and writes PostgreSQL; JSON goes back to the browser.

LWT - Learning Platform

1 **User** Student / Teacher → 2 **Django** MVT → 3 **PostgreSQL** Database → 4 **Media** Uploads → 5 **Render** Deploy

Teachers upload study material (media files); students access it through role-based views; Render hosts the Django app.

VBI - Virtual Bank of India

1 **User** Customer → 2 **Django** Backend → 3 **PostgreSQL** Database → 4 **Transactions** Secure → 5 **Render** Deploy

Each transfer is one atomic database transaction with row locking; sensitive endpoints require authentication and permission checks.

Safe Money Transfer (VBI)

```
from django.db import transaction

@transaction.atomic
def transfer(sender_id, receiver_id, amount):
    s = Account.objects.select_for_update().get(pk=sender_id)
    r = Account.objects.select_for_update().get(pk=receiver_id)
    if s.balance < amount:
        raise ValueError("Insufficient funds")
    s.balance -= amount
    r.balance += amount
    s.save()
    r.save()
```

Important Notes

- **Why PostgreSQL?** Better concurrency, constraints and production readiness than SQLite.
- **Uploads on Render:** the disk is ephemeral, so use cloud storage (S3 / Cloudinary) for media.
- **Scaling first steps:** DB indexes, pagination, caching (Redis).

8 Final Tips & Next Steps

Stay confident and be your best self!

1. **Revise core concepts**
   Focus on your resume skills and projects.
2. **Practise coding**
   Solve problems on HackerRank / LeetCode daily.
3. **Explain your projects**
   Know the flow, tech stack and challenges.
4. **Prepare for HR round**
   Study common questions and be confident.
5. **Stay positive**
   Good preparation builds confidence.

Interview Day Checklist

- Resume copies, ID and notebook ready
- Test camera, mic and internet (online interview)
- Join 10 minutes early
- Keep water nearby and a calm pace
- Listen to the full question before answering
- Send a short thank-you note afterwards

Common Mistakes to Avoid

- Memorising answers without understanding
- Not knowing your own project details
- Staying silent when stuck (think aloud!)
- Over-claiming skills you do not have
- Speaking badly about past teams or employers
- Skipping questions to ask the interviewer

Smart Questions to Ask the Interviewer

- What does a typical day or first project look like for this role?
- Which tech stack and tools does the team use, and how are code reviews done?
- How does the company support learning and growth for developers?

You've got this!

Hard Work + Smart Preparation = Success